4 documents to organise your app build

USER-FLOWS.md • ENV.md • CHANGELOG.md • TASKS.md

A beginner-friendly guide with complete sample files, explanations and instructions for adapting them to your project.

How to use this guide

• Create a docs folder in your project. Add the four files using the exact filenames shown here.

• Copy the sample file text from each lavender box. Replace TaskFlow and its details with your own app information.

• Use the guidance page after each sample to understand what to change. Remove features or services your app does not use.

• Ask your coding agent to read the relevant files before working. Review any updates it makes to these documents.

One example app throughout

TaskFlow is a simple web app for freelance designers to create tasks, set due dates and mark work complete. The samples describe a fictional project; adapt them to your actual implementation.

Keep the files connected

USER-FLOWS describes the experience. ENV describes configuration. TASKS tracks planned work. CHANGELOG records changes that actually shipped.

Where the files go

```
my-app/
docs/
USER-FLOWS.md
ENV.md
CHANGELOG.md
TASKS.md
.env.example
.gitignore
```

The .env.example and .gitignore files are configuration companions, not extra documentation files. ENV.md explains them; it does not replace them.

USER-FLOWS.md

Maps how users complete tasks, including the paths they take when something goes wrong.

```
# User Flows - TaskFlow

## Purpose and audience
Help freelance designers track client tasks and deadlines.
Main outcome: know what to do next and track completion.

## Flow 1: Create a task
Entry: Dashboard.
1. Select New task.
2. Enter a title and optional due date.
3. Select Save.
4. The saved task appears in the task list.
Success: close the form and show "Task created".
Validation: a blank title shows "Enter a task title".
Save failure: keep the input and offer Retry.
Cancel: close the form without saving.

## Flow 2: Complete a task
Entry: Task list.
1. Select the task's completion control.
2. Mark the task complete and update its appearance.
3. Keep the completed status after refresh.
Failure: restore the previous status and show Retry.

## Flow 3: View tasks
Entry: Dashboard.
Loading: show a loading indicator while fetching tasks.
Empty: show "No tasks yet" and a Create task button.
Error: show "Unable to load tasks" and a Retry button.
Success: show task titles, due dates and completion status.

## Navigation and accessibility
Users can return to the dashboard from every task view.
Forms work with a keyboard and have labelled fields.
Feedback does not rely on colour alone.
On mobile, controls and text remain usable.

## Open decisions
Confirm whether accounts are needed before adding login.
Confirm where tasks are stored before implementation.
```

Make the user flows yours

What each section contains

• Purpose and audience: who uses the app and the main result they need.

• Flow: one user goal, where it starts, numbered actions and the result.

• Success and failures: what the user sees, what happens to their input and how they recover.

• Navigation and accessibility: how users move around and operate controls.

• Open decisions: questions that need an answer before the agent implements a feature.

Start small

Map the 2-3 most important journeys. For a store, that might be find a product, add to cart and checkout. For a booking app, it could be select a service, choose a date and confirm.

Before approving the file

• Every flow has an entry point and a clear outcome.

• Required inputs and validation are described.

• Loading, empty, error and success states are covered where relevant.

• Mobile use and back navigation are considered.

• Unknown details are recorded instead of silently assumed.

Instruction for your agent

Read docs/USER-FLOWS.md and compare it with the current app. Identify missing screens, actions and feedback states. Ask about open decisions before implementing them. Update the document only to reflect approved behaviour.

Update when

A journey, screen, form, validation rule or navigation path changes. Keep the file aligned with the actual experience.

ENV.md

Explains the variables and setup needed to run the app. The example assumes server-side database access.

```
# Environment Setup - TaskFlow

## Required variables
APP_URL
- Purpose: the app's base URL.
- Local example: http://localhost:3000
- Production: use the deployed app URL.

DATABASE_URL
- Purpose: server-side database connection.
- Obtain from: your database provider's connection settings.
- Example: <your-database-connection-string>
- Secret: yes. Never expose it in browser code.

## Local setup
1. Copy .env.example to .env.local if the framework supports it.
Otherwise use the environment filename it expects.
2. Replace placeholder values with your own configuration.
3. Install dependencies and run the project's start command.
4. Restart the app after changing environment variables.
5. Verify that the server can connect to the database.

## Example file: .env.example
APP_URL=http://localhost:3000
DATABASE_URL=<your-database-connection-string>

## Git exclusions: .gitignore
.env
.env.*
!.env.example

## Preview and production
Set variables in the hosting provider's environment settings.
Use separate test credentials for preview where possible.
Do not use production data for routine local testing.
Redeploy or restart as required after changing values.

## Validation and secret handling
Fail clearly when a required variable is missing.
Log variable names, never secret values.
Only use framework public prefixes for non-secret values.
Commit .env.example with placeholders, never real secrets.
If a secret was exposed, revoke or rotate it.
```

Document configuration safely

What is an environment variable?

A setting your app reads from its environment rather than hardcoding it into source code. Some values are public configuration; others, such as database passwords, are secrets.

What each section contains

• Required variables: exact names, purpose, whether they are secret and where to obtain them.

• Local setup: the actual filename and commands your project uses. Frameworks load environment files differently.

• Example file: safe sample values that someone can copy to get started.

• Git exclusions: rules that keep local secret files out of version control.

• Preview and production: where deployed values are configured.

• Validation: how the app checks missing settings without revealing secrets.

Adapt the sample

Remove DATABASE_URL if your app does not use it. Add variables only for services your implementation actually needs. Confirm variable names against the code. APP_URL is a sample name, not a universal requirement.

Before approving the file

• Every required variable is documented using its exact name.

• No real keys, passwords or connection strings appear in the docs or example file.

• The environment filename and setup commands match the project.

• Secrets are used only on the server.

• Hosting settings and local settings are distinguished.

Instruction for your agent

Inspect the configuration used by this project without printing secret values. Draft docs/ENV.md and .env.example using placeholders. Explain which variables are required, which are server-only and how to configure local and deployed environments. Verify the .gitignore rules.

CHANGELOG.md

A history of shipped changes. These fictional entries show the structure; replace them with your real releases.

```
# Changelog - TaskFlow

Record user-facing changes. Keep the newest release first.
Use Added, Changed, Fixed and Removed when relevant.

## [Unreleased]
### Added
- Planned: a filter for completed tasks.

## [1.2.0] - 2026-10-08
### Added
- Search tasks by title.
### Changed
- Show overdue dates more clearly in the task list.
### Fixed
- Prevent duplicate tasks when Save is clicked twice.

## [1.1.0] - 2026-10-01
### Added
- Optional due dates when creating tasks.
### Fixed
- Keep the completed status after refreshing the page.

## [1.0.0] - 2026-09-25
### Added
- Create tasks with a required title.
- View saved tasks in the dashboard.
- Mark tasks complete.

## Release rules
- Move shipped entries from Unreleased to a dated release.
- Use the version or release label actually used by the project.
- Explain the effect on users, not just internal file changes.
- Record breaking changes and required user actions clearly.
- Never include secrets or private customer information.
```

Example of a useful entry

“Fixed duplicate tasks when Save is clicked twice” explains the trigger and outcome. “Fixed bugs” does not.

Keep a useful change history

What each section contains

• Unreleased: changes being prepared, clearly separated from published releases. Track detailed work in TASKS.md.

• Release heading: your actual version or release label and release date.

• Added: new features people can use.

• Changed: existing behaviour that now works differently.

• Fixed: a specific issue that was resolved.

• Removed: functionality that is no longer available, when applicable.

Do not copy fictional release history

Start with your real first release. If the app has not launched, use Unreleased. You do not need every category in every release; omit empty ones.

Before approving the file

• Published entries describe changes that actually shipped.

• Dates and release labels are accurate.

• The newest release appears first.

• Each entry explains a feature or a concrete user impact.

• Breaking changes explain what users need to do.

Instruction for your agent

Review the approved changes in this release. Draft concise entries for docs/CHANGELOG.md grouped by Added, Changed, Fixed or Removed. Do not invent dates, version numbers or shipped features. Put unshipped changes under Unreleased and ask me to confirm the release details.

Update when

Preparing a release and after confirming what shipped. Use TASKS.md for ongoing task status; use CHANGELOG.md for the release history.

TASKS.md

Breaks the build into reviewable work with dependencies and a clear definition of completion.

```
# Build Tasks - TaskFlow

## Goal and scope
Build a task app for freelance designers.
First version: create, view and complete tasks with due dates.
Later: collaboration, reminders and advanced reporting.

## Status rules
[ ] Not verified complete. [x] Implemented and verified.
Use a Status line to show To do, In progress or Blocked.

## TASK-01: Set up the project
- [ ] Status: To do
Priority: High. Dependencies: none.
Done when: the app runs locally and opens the dashboard.
Check: follow the README setup from a fresh checkout.

## TASK-02: Store and list tasks
- [ ] Status: To do
Priority: High. Dependencies: TASK-01.
Data: title, optional due date and completion status.
Done when: saved tasks appear and persist after refresh.
Check: create test data, refresh and verify the list.

## TASK-03: Create a task
- [ ] Status: To do
Priority: High. Dependencies: TASK-02.
Done when: a valid title saves one task; blank titles fail.
Save failure preserves input and offers Retry.
Check: valid save, blank title and failed save.

## TASK-04: Complete a task
- [ ] Status: To do
Priority: High. Dependencies: TASK-02.
Done when: completion updates and persists after refresh.
Failure restores the prior state and shows a retry option.
Check: complete a task, refresh and test a failed update.

## TASK-05: Review the core flows
- [ ] Status: To do
Priority: High. Dependencies: TASK-03 and TASK-04.
Done when: core flows work on mobile and desktop.
Check: keyboard use, loading, empty, error and success states.

## Blockers and next action
Record the blocker, the decision needed and the next task.
Do not mark a task complete until its checks pass.
```

Make tasks easy to review

What each section contains

• Goal and scope: what this version includes and what is deferred.

• Task ID and title: a stable reference and one clear outcome.

• Status: To do, In progress, Blocked or Done. Check the box only after verification.

• Priority: what matters first. Keep dependencies consistent with the build order.

• Dependencies: tasks that must be completed before this task can proceed.

• Done when: specific acceptance criteria, not “looks good”.

• Check: how you or the agent will verify the result.

• Blockers: unresolved information and the next action needed.

Keep each task small

“Build the entire dashboard” is often too large. Split it into display tasks, create a task, complete a task and handle empty/error states. Add relevant file paths only after the project structure is known.

Before approving the file

• Tasks describe approved features, not extra ideas invented by the agent.

• Each task has one outcome and testable completion criteria.

• Dependencies allow a sensible implementation order.

• Blocked work includes the decision needed to continue.

• Completed work has been checked, not merely generated.

Instruction for your agent

Read the project requirements and docs/USER-FLOWS.md. Create or update docs/TASKS.md with small tasks, dependencies, priorities and completion checks. Work on the next unblocked task within the approved scope. Report the result and verification before marking it complete.

PUT IT INTO PRACTICE

Your document workflow

Before building

Write the core user flows. Confirm scope and convert it into tasks. Document the configuration your actual stack needs.

During building

Ask your agent to read the relevant files. Work through one task at a time, review the result and update the status. Revise flows when you approve a behaviour change.

Before a release

Check the core journeys, verify deployed configuration and record the changes that are actually shipping. Confirm the date and release label.

Reusable instruction

Read docs/USER-FLOWS.md, docs/ENV.md and docs/TASKS.md before making changes. Summarise the next task and any blockers. Keep implementation within the approved scope. Use placeholders when documenting configuration. Update task status only after verification. Draft CHANGELOG entries for approved changes without claiming they shipped until the release is confirmed.

Final document check

- The documents describe this app, not the sample app.
- No real secrets are included.
- User flows match approved behaviour.
- Environment variables match the implementation.
- Tasks have clear checks and accurate statuses.
- Release history contains real changes and correct dates.

Simple is enough

These filenames are useful project conventions, not mandatory standards. Keep the content short, accurate and useful. Add detail when the project needs it.